

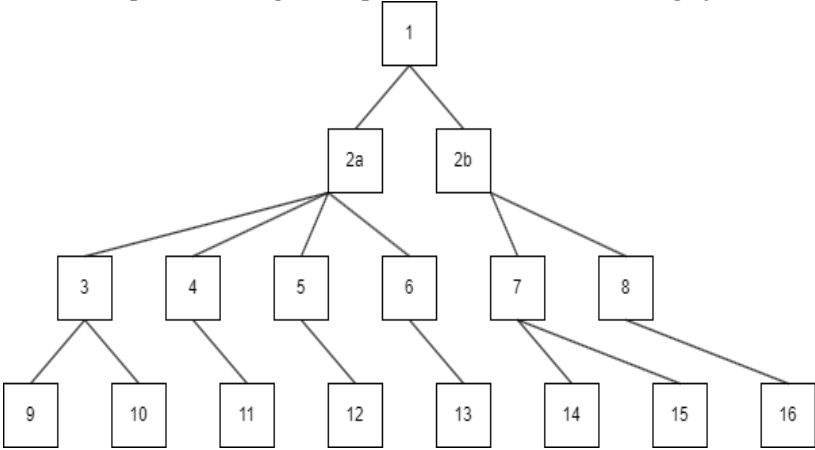


NATIONAL INSTITUTE OF TECHNOLOGY ROURKELA-769008
END-SEMESTER EXAMINATION, 2023
SESSION: 2022 – 2023 (Spring)
 B. Tech. 8th / MTech. 2nd Semester

Subject code: CS6404 **Subject Name:** Software Testing
No. of pages: 3 **Full Marks:** 100

Dept. Code: CS
Duration: 3 Hours

Figures at the right-hand margin indicate marks.
All parts of a question should be answered at one place.
Answer any FIVE questions including Q1.

Q. No.	Particulars	Marks
1 (a)	What will be the total number of test sessions required using neighborhood integration testing, if a call graph has total 15 nodes, out of which 5 nodes are sink nodes?	2x10=20
(b)	If a test suite T contains 100 tests of which 20 are non-modification-revealing for program P and modified program P', and the technique M omits 10 of these 20 tests, what is the precision of M relative to P, P' and T?	
(c)	Which UML diagrams are most suitable for unit testing of object-oriented software?	
(d)	Calculate the fault exposing potential of a test case, if a program has 5 mutants and 3 gets killed by the test case.	
(e)	What do you mean by smoke testing? Briefly explain it in the context of Banking Information System.	
(f)	What do you mean by test incident report? Why it is used?	
(g)	What is the implication of multiple inheritance in testing of object-oriented software?	
(h)	For integration testing, which test design pattern will be most suitable?	
(i)	Mention two types of faults that are encountered in fault-based testing, along with suitable examples.	
(j)	Mention the name of a tool that is used for regression testing.	
2(a)	Discuss how you would perform system testing of a software that implements a bounded queue of positive integral elements. Assume that the queue supports only the functions insert an element, delete an element, and find an element.	10+10=20
(b)	Perform top-down integration procedure for the following system hierarchy.  <pre> graph TD 1[1] --> 2a[2a] 1 --> 2b[2b] 2a --> 3[3] 2a --> 4[4] 2a --> 5[5] 2a --> 6[6] 2b --> 7[7] 2b --> 8[8] 3 --> 9[9] 3 --> 10[10] 4 --> 11[11] 5 --> 12[12] 6 --> 13[13] 7 --> 14[14] 7 --> 15[15] 8 --> 16[16] </pre>	

	<table><tr><td>Problem ID</td><td>Potential Problems</td><td>Uncertainty Factor</td><td>Risk Impact</td></tr><tr><td>P1</td><td>Requirement Specification ambiguity</td><td>2</td><td>3</td></tr><tr><td>P2</td><td>Database Interface problems</td><td>5</td><td>6</td></tr><tr><td>P3</td><td>File corruption</td><td>6</td><td>4</td></tr><tr><td>P4</td><td>Databases not synchronized</td><td>8</td><td>7</td></tr><tr><td>P5</td><td>Unavailability of modules for integration</td><td>9</td><td>10</td></tr><tr><td>P6</td><td>Graphical User Interface problems</td><td>4</td><td>5</td></tr><tr><td>P7</td><td>Test cases not ready</td><td>7</td><td>9</td></tr></table> Prioritize the potential problems of the application using risk-based prioritization technique.	Problem ID	Potential Problems	Uncertainty Factor	Risk Impact	P1	Requirement Specification ambiguity	2	3	P2	Database Interface problems	5	6	P3	File corruption	6	4	P4	Databases not synchronized	8	7	P5	Unavailability of modules for integration	9	10	P6	Graphical User Interface problems	4	5	P7	Test cases not ready	7	9	
Problem ID	Potential Problems	Uncertainty Factor	Risk Impact																															
P1	Requirement Specification ambiguity	2	3																															
P2	Database Interface problems	5	6																															
P3	File corruption	6	4																															
P4	Databases not synchronized	8	7																															
P5	Unavailability of modules for integration	9	10																															
P6	Graphical User Interface problems	4	5																															
P7	Test cases not ready	7	9																															
5(a)	Consider a Library Information System with the following functionalities. <table><tr><td>Serial Number</td><td>Functionality</td><td>Function ID in SRS</td><td>Test Cases</td></tr><tr><td>1</td><td>Login the system</td><td>F 3.4</td><td>T1</td></tr><tr><td>2</td><td>Check availability of books</td><td>F 3.5</td><td>T2</td></tr><tr><td>3</td><td>Reserve book</td><td>F 3.6</td><td>T3</td></tr><tr><td>4</td><td>Issue book</td><td>F 3.7</td><td>T4</td></tr><tr><td>5</td><td>Return book</td><td>F 3.8</td><td>T5</td></tr><tr><td>6</td><td>Renew book</td><td>F 3.9</td><td>T6</td></tr><tr><td>7</td><td>Exit</td><td>F 3.10</td><td>T7</td></tr></table> Develop the test specifications and test log for Return book functionality.	Serial Number	Functionality	Function ID in SRS	Test Cases	1	Login the system	F 3.4	T1	2	Check availability of books	F 3.5	T2	3	Reserve book	F 3.6	T3	4	Issue book	F 3.7	T4	5	Return book	F 3.8	T5	6	Renew book	F 3.9	T6	7	Exit	F 3.10	T7	10+10=20
Serial Number	Functionality	Function ID in SRS	Test Cases																															
1	Login the system	F 3.4	T1																															
2	Check availability of books	F 3.5	T2																															
3	Reserve book	F 3.6	T3																															
4	Issue book	F 3.7	T4																															
5	Return book	F 3.8	T5																															
6	Renew book	F 3.9	T6																															
7	Exit	F 3.10	T7																															
(b)	Suppose you are given the task of developing a web application for student admission in NIT Rourkela. Briefly explain the following types of web testing that you need to perform for the above web application. (i) Content testing (ii) Navigation testing (iii) Compatibility testing																																	
6(a)	What are the different types of errors that integration testing targets to detect? Give two examples of such errors. Explain the different integration testing strategies for object-oriented software.	10+10=20																																
(b)	What do you understand by test coverage analysis? Why the traditional coverages like statement coverage cannot be used for object-oriented software? Define any four test coverage criteria that can be used explicitly during testing of object-oriented software, using UML diagrams.																																	
7(a)	Briefly explain the test procedure using Extended Use Case test pattern. Apply the above test procedure to the Cash Withdrawal use case of ATM system, and develop the test cases in a tabular manner.	10+10=20																																
(b)	Consider the Book class in a Library Information System. Perform random class testing on the Book class and generate at least 5 test sequences.																																	